

Class Work Linux Lab: Files, APT, and C Programming

Biraj Sharma Chapagain (250125)
BSc (Hons) Computer Science, Softwarica College of IT & E-commerce.
ST5003CMD Operating Systems, Security, and Networks
Submitted to Rupak Rajbanshi
Date: 8th September 2026

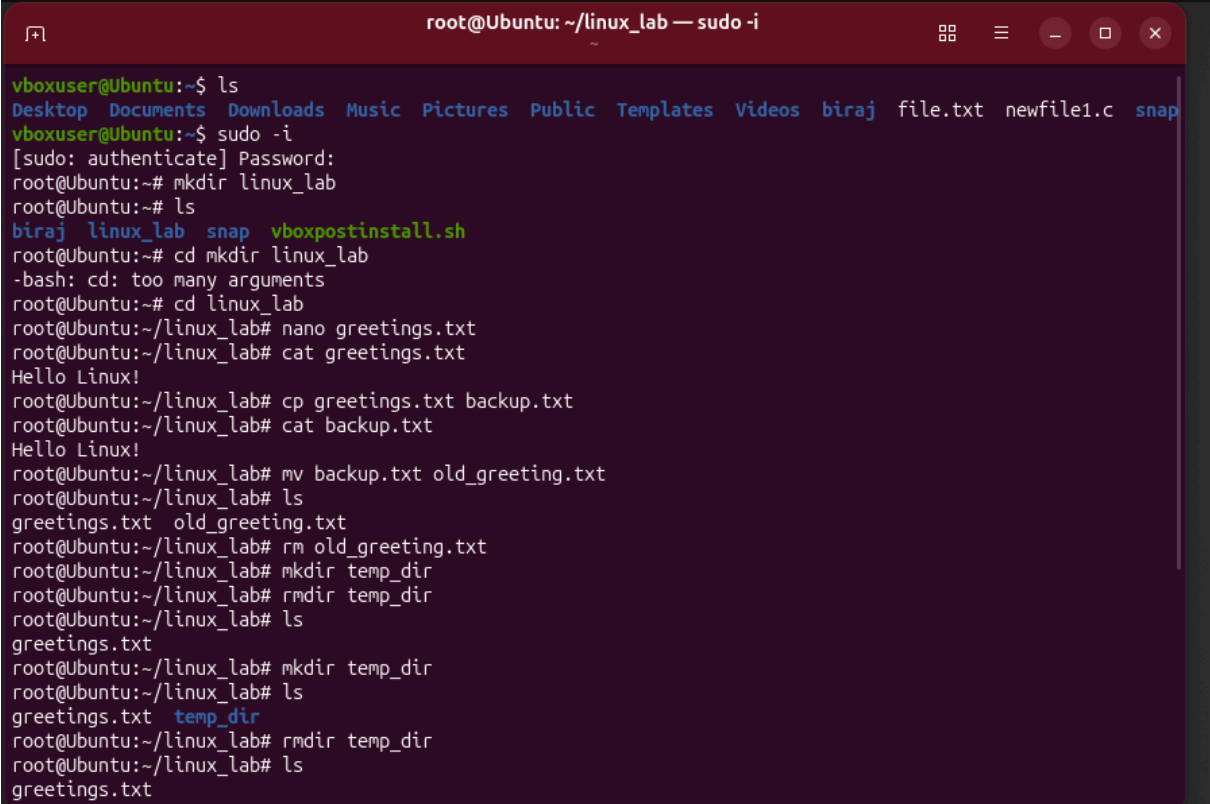
Table of content:

Task No.1 - FILE OPERATIONS:	3
Task No.2 - APT PACKAGE MANAGEMENT:	5
Task No.3 - C COMPILATION	7
Step 1: Writing the code.	7
Step 2: Compilation and Run.	8
Step 3: Modification and Recompilation.	9
Step 4: Cleanup & Verification	10

Table of Figures:

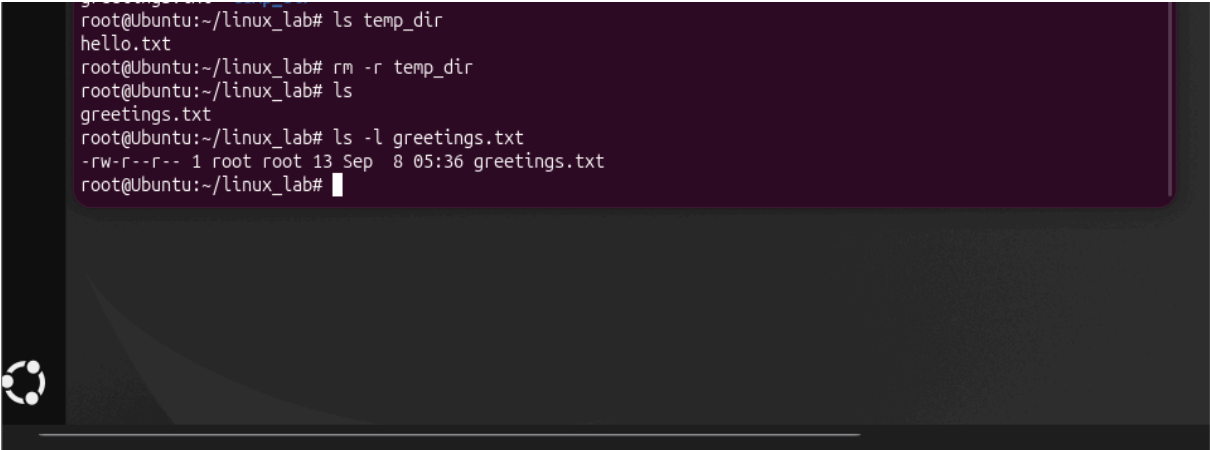
Figure 1: Essential file commands.	4
Figure 2: Commands continue.	4
Figure 3 : apt commands	6
Figure 4: apt commands continue	7
Figure 5: Creating and editing the source file.	8
Figure 6: Code compilation and execution through commands.	9
Figure 7:Iterative Development	10
Figure 8: Final Checks	11

Task No.1 - FILE OPERATIONS:

A terminal window titled 'root@Ubuntu: ~/linux_lab — sudo -i'. The user 'vboxuser@Ubuntu' runs 'ls' showing a directory listing. Then 'sudo -i' is used to become root. Root runs 'mkdir linux_lab' and 'ls' to confirm. Then 'cd linux_lab' is attempted but fails with '-bash: cd: too many arguments'. Then 'cd /linux_lab' is used. Root runs 'nano greetings.txt', 'cat greetings.txt' (output: 'Hello Linux!'), 'cp greetings.txt backup.txt', 'cat backup.txt' (output: 'Hello Linux!'), 'mv backup.txt old_greeting.txt', 'ls' (output: 'greetings.txt old_greeting.txt'), 'rm old_greeting.txt', 'mkdir temp_dir', 'rmdir temp_dir', 'ls' (output: 'greetings.txt'), 'mkdir temp_dir', 'ls' (output: 'greetings.txt temp_dir'), 'rmdir temp_dir', 'ls' (output: 'greetings.txt').

```
root@Ubuntu: ~/linux_lab — sudo -i
vboxuser@Ubuntu:~$ ls
Desktop  Documents  Downloads  Music  Pictures  Public  Templates  Videos  biraj  file.txt  newfile1.c  snap
vboxuser@Ubuntu:~$ sudo -i
[sudo: authenticate] Password:
root@Ubuntu:~# mkdir linux_lab
root@Ubuntu:~# ls
biraj  linux_lab  snap  vboxpostinstall.sh
root@Ubuntu:~# cd mkdir linux_lab
-bash: cd: too many arguments
root@Ubuntu:~# cd linux_lab
root@Ubuntu:~/linux_lab# nano greetings.txt
root@Ubuntu:~/linux_lab# cat greetings.txt
Hello Linux!
root@Ubuntu:~/linux_lab# cp greetings.txt backup.txt
root@Ubuntu:~/linux_lab# cat backup.txt
Hello Linux!
root@Ubuntu:~/linux_lab# mv backup.txt old_greeting.txt
root@Ubuntu:~/linux_lab# ls
greetings.txt  old_greeting.txt
root@Ubuntu:~/linux_lab# rm old_greeting.txt
root@Ubuntu:~/linux_lab# mkdir temp_dir
root@Ubuntu:~/linux_lab# rmdir temp_dir
root@Ubuntu:~/linux_lab# ls
greetings.txt
root@Ubuntu:~/linux_lab# mkdir temp_dir
root@Ubuntu:~/linux_lab# ls
greetings.txt  temp_dir
root@Ubuntu:~/linux_lab# rmdir temp_dir
root@Ubuntu:~/linux_lab# ls
greetings.txt
```

Figure 1: Essential file commands.

A terminal window showing the continuation of file operations. Root runs 'ls temp_dir' (output: 'hello.txt'), 'rm -r temp_dir', 'ls' (output: 'greetings.txt'), 'ls -l greetings.txt' (output: '-rw-r--r-- 1 root root 13 Sep 8 05:36 greetings.txt'), and 'ls' again.

```
root@Ubuntu:~/linux_lab# ls temp_dir
hello.txt
root@Ubuntu:~/linux_lab# rm -r temp_dir
root@Ubuntu:~/linux_lab# ls
greetings.txt
root@Ubuntu:~/linux_lab# ls -l greetings.txt
-rw-r--r-- 1 root root 13 Sep  8 05:36 greetings.txt
root@Ubuntu:~/linux_lab#
```

Figure 2: Commands continue.

Summary: We performed tasks like making a directory using `mkdir_file-name`, listing files/ folders using `ls`, and creating a file "greetings.txt" using nano command.

Task No.2 - APT PACKAGE MANAGEMENT:

```
root@Ubuntu:~/linux_lab# gcc --version
gcc (Ubuntu 15.2.0-16ubuntu1) 15.2.0
Copyright (C) 2025 Free Software Foundation, Inc.
This is free software; see the source for copying conditions. There is NO
warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.

root@Ubuntu:~/linux_lab# apt search python3
alenbic/resolute 1.18.4-1 all
  lightweight database migration tool for SQLAlchemy

antimony/resolute 0.9.3-2.1build2 amd64
  Computer-aided design CAD tool

astro-frameworks/resolute 5.0 all
  Astronomical data analysis frameworks

astro-python3/resolute 5.0 all
  Python 3 packages for astronomy

bamkit/resolute 0.0.1+git20170413.ccd079d-3build1 all
  tools for common BAM file manipulations

beancount/resolute 3.2.0-2build1 all
  Double-entry accounting from text files

berrynet-dashboard/resolute 3.10.2-2build1 all
  deep learning gateway - python3 modules
```

Figure 3 : apt commands

```

root@Ubuntu:~/linux_lab# sudo apt install build-essential
The following packages were automatically installed and are no longer required:
  linux-headers-7.0.0-30          linux-modules-7.0.0-30-generic
  linux-headers-7.0.0-30-generic  linux-tools-7.0.0-30
  linux-image-unsigned-7.0.0-30-generic  linux-tools-7.0.0-30-generic
  linux-main-modules-zfs-7.0.0-30-generic
Use 'sudo apt autoremove' to remove them.

Installing:
  build-essential

Installing dependencies:
  bzip2      g++-15      libalgorithm-diff-xs-perl  libfakeroot      make
  dpkg-dev  g++-15-x86-64-linux-gnu  libalgorithm-merge-perl  libfile-fcntllock-perl
  fakeroot  g++-x86-64-linux-gnu     libcrypt-dev             libstdc++-15-dev
  g++       libalgorithm-diff-perl   libdpkg-perl             lto-disabled-list

Suggested packages:
  bzip2-doc      debian-tag2upload-keyring  g++-15-multilib  git  libstdc++-15-doc
  debian-keyring  g++-multilib              gcc-15-doc       bzip  make-doc

Summary:
  Upgrading: 0, Installing: 18, Removing: 0, Not Upgrading: 0
  Download size: 19.3 MB
  Space needed: 73.4 MB / 32.5 GB available

Continue? [Y/n] y

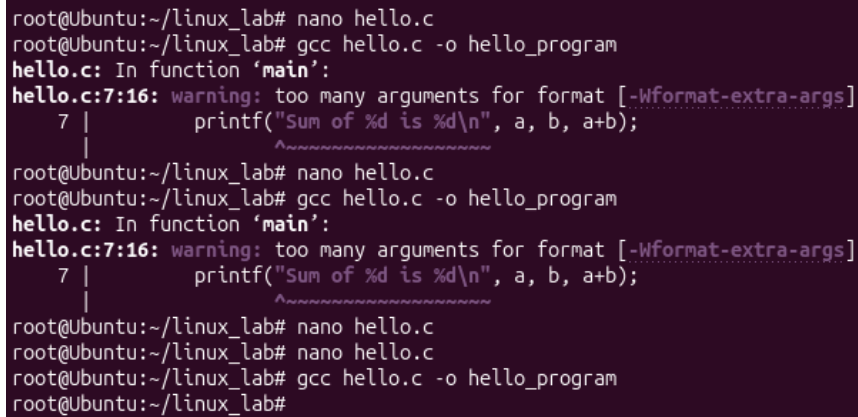
```

Figure 4: apt commands continue

Summary: Update of pre existing apts and Installation of new tools required for the further tasks were done using the Commands like apt update, sudo apt install build-essential, gcc --version and searching for other packages like python3 using “apt search python3”

Task No.3 - C COMPILATION

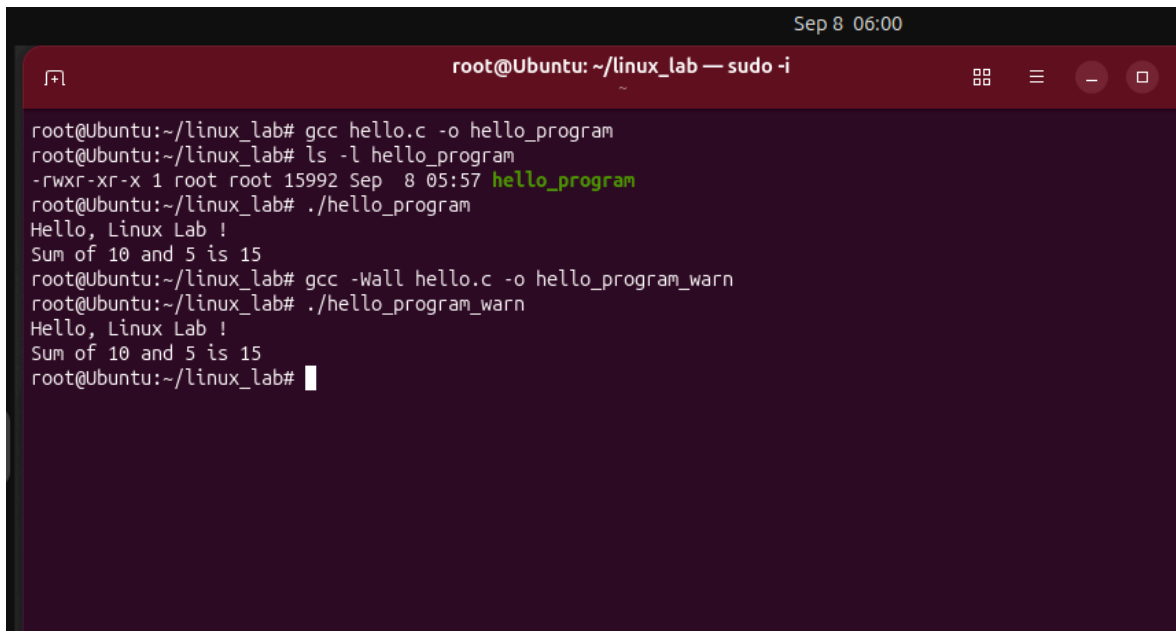
Step 1: Writing the code.



```
root@Ubuntu:~/linux_lab# nano hello.c
root@Ubuntu:~/linux_lab# gcc hello.c -o hello_program
hello.c: In function 'main':
hello.c:7:16: warning: too many arguments for format [-Wformat-extra-args]
   7 |         printf("Sum of %d is %d\n", a, b, a+b);
     |
root@Ubuntu:~/linux_lab# nano hello.c
root@Ubuntu:~/linux_lab# gcc hello.c -o hello_program
hello.c: In function 'main':
hello.c:7:16: warning: too many arguments for format [-Wformat-extra-args]
   7 |         printf("Sum of %d is %d\n", a, b, a+b);
     |
root@Ubuntu:~/linux_lab# nano hello.c
root@Ubuntu:~/linux_lab# nano hello.c
root@Ubuntu:~/linux_lab# gcc hello.c -o hello_program
root@Ubuntu:~/linux_lab#
```

Figure 5: Creating and editing the source file.

Step 2: Compilation and Run.

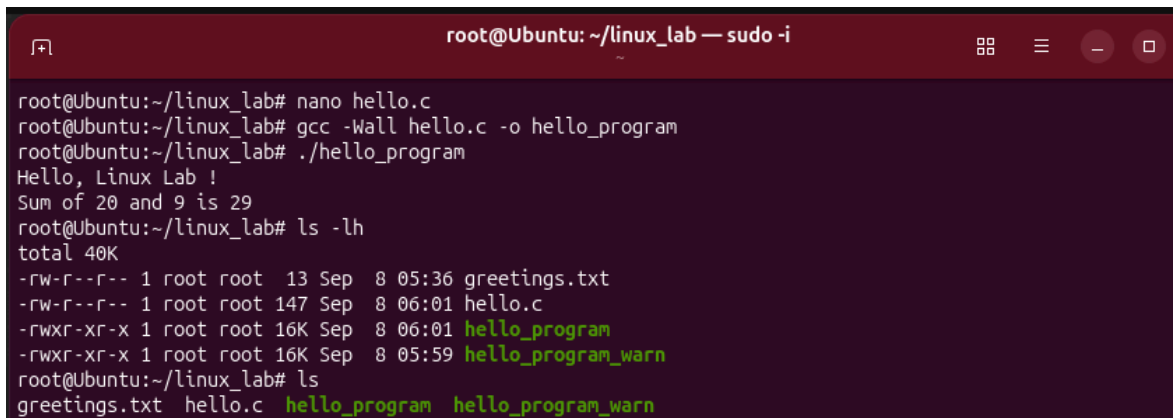
A terminal window titled "root@Ubuntu: ~/linux_lab — sudo -i" with a timestamp "Sep 8 06:00". The terminal shows the following commands and output:

```
root@Ubuntu:~/linux_lab# gcc hello.c -o hello_program
root@Ubuntu:~/linux_lab# ls -l hello_program
-rwxr-xr-x 1 root root 15992 Sep  8 05:57 hello_program
root@Ubuntu:~/linux_lab# ./hello_program
Hello, Linux Lab !
Sum of 10 and 5 is 15
root@Ubuntu:~/linux_lab# gcc -Wall hello.c -o hello_program_warn
root@Ubuntu:~/linux_lab# ./hello_program_warn
Hello, Linux Lab !
Sum of 10 and 5 is 15
root@Ubuntu:~/linux_lab#
```

Figure 6: Code compilation and execution through commands.

Summary: The command “nano” was used to create and edit a C-file and then the rest of the commands were used to specify the executable name, Check if the executable was created, running the compiled program and a file with Compilation warnings enabled was created respectively.

Step 3: Modification and Recompilation.



```
root@Ubuntu: ~/linux_lab — sudo -i
root@Ubuntu:~/linux_lab# nano hello.c
root@Ubuntu:~/linux_lab# gcc -Wall hello.c -o hello_program
root@Ubuntu:~/linux_lab# ./hello_program
Hello, Linux Lab !
Sum of 20 and 9 is 29
root@Ubuntu:~/linux_lab# ls -lh
total 40K
-rw-r--r-- 1 root root 13 Sep  8 05:36 greetings.txt
-rw-r--r-- 1 root root 147 Sep  8 06:01 hello.c
-rwxr-xr-x 1 root root 16K Sep  8 06:01 hello_program
-rwxr-xr-x 1 root root 16K Sep  8 05:59 hello_program_warn
root@Ubuntu:~/linux_lab# ls
greetings.txt  hello.c  hello_program  hello_program_warn
```

Figure 7: Iterative Development

Summary: The above commands were used to edit the file to change the numbers, recompile the code in the file and run to verify the changes.

Step 4: Cleanup & Verification

A terminal window titled 'root@Ubuntu: ~ — sudo -i' with standard window controls. The terminal shows a series of commands and their outputs. The user navigates to the home directory, lists files in the ~/linux_lab directory with long format, cat's the contents of hello.c, removes the entire ~/linux_lab directory, and lists the root directory to verify its removal. The output of the final 'ls' command shows 'biraj', 'snap', and 'vboxpostinstall.sh' in the root directory.

```
root@Ubuntu:~# cd ~
root@Ubuntu:~# ls -lh ~/linux_lab/
total 40K
-rw-r--r-- 1 root root 13 Sep  8 05:36 greetings.txt
-rw-r--r-- 1 root root 147 Sep  8 06:01 hello.c
-rwxr-xr-x 1 root root 16K Sep  8 06:01 hello_program
-rwxr-xr-x 1 root root 16K Sep  8 05:59 hello_program_warn
root@Ubuntu:~# cat ~/linux_lab/hello.c
#include <stdio.h>

int main(){
    int a = 20, b=9;

    printf("Hello, Linux Lab !\n");
    printf("Sum of %d and %d is %d\n", a, b, a+b);

    return 0;
}
root@Ubuntu:~# rm -rf ~/linux_lab
root@Ubuntu:~# ls
biraj  snap  vboxpostinstall.sh
root@Ubuntu:~#
```

Figure 8: Final Checks

Summary: The commands were used to navigate to the home directory, listing all files in the lab directory with details and performing the final verifications.